

Lecture 9: Latches, Flip-Flops, and Registers

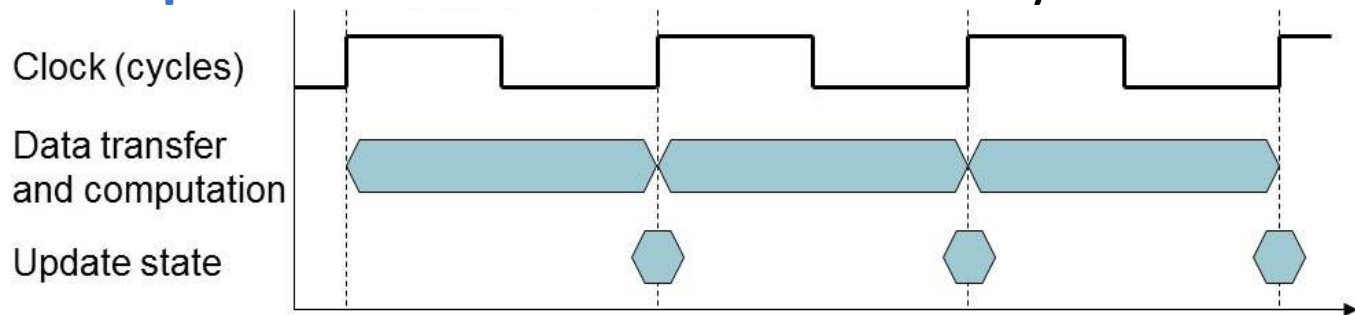
CMPS 221 – Computer Organization and Design

Recall: Types of Logic Blocks

- The two general categories of logic blocks are:
 - **Combinational Logic**: output depends only on the current input
 - **Sequential Logic**: output depends on both the current input as well as the values stored in the block (memory) } focus of today

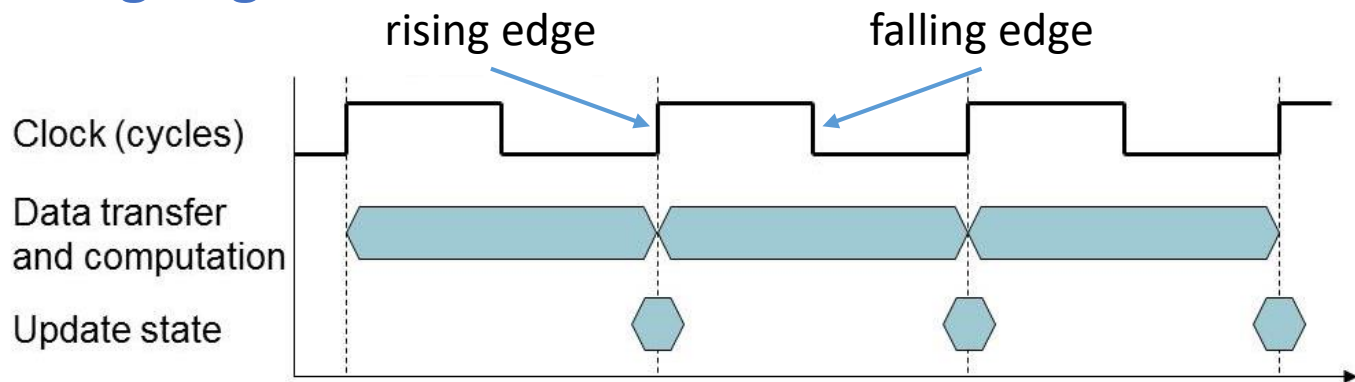
Recall: Clocks

- A computer is driven by a **clock** that determines when events take place
 - A clock **cycle** is a discrete time interval between two pulses of an oscillator
 - A clock **period** is the duration of a clock cycle



Edge-triggered Clocking

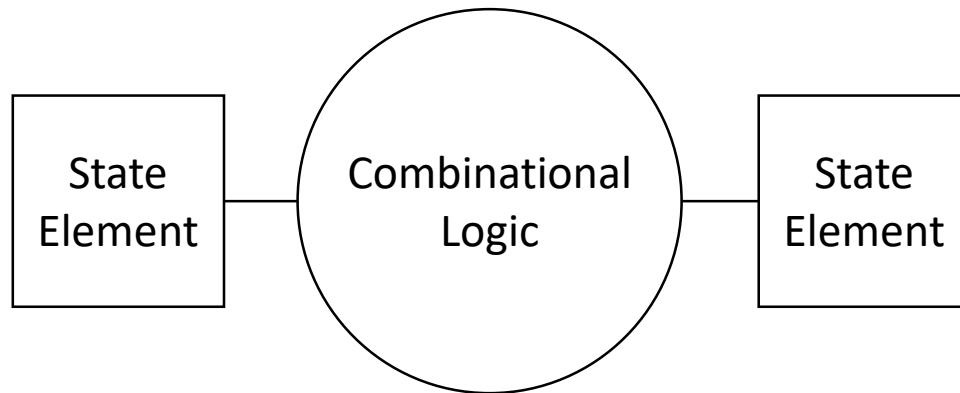
- A clock **edge** is a transition of the clock between two values
 - **Rising edge**: transition from 0 to 1
 - **Falling edge**: transition from 1 to 0



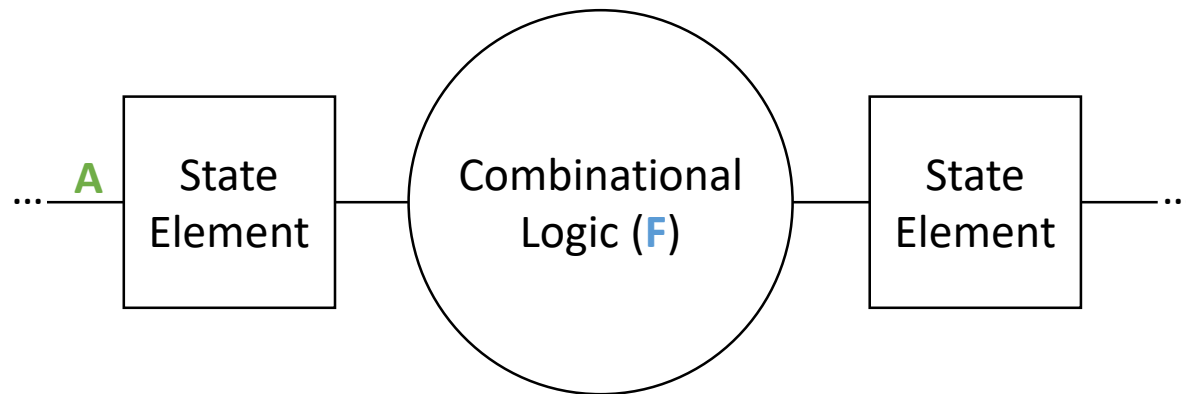
- **Edge-triggered clocking** is a methodology where either the rising or falling edge of a clock is *active* and all updates to state happen on that edge

State Elements

- In edge-triggered clocking, a **state element** is a circuit component whose content (state) only changes on the active clock edge
- Sequential logic can be built by combining combinational logic with state elements

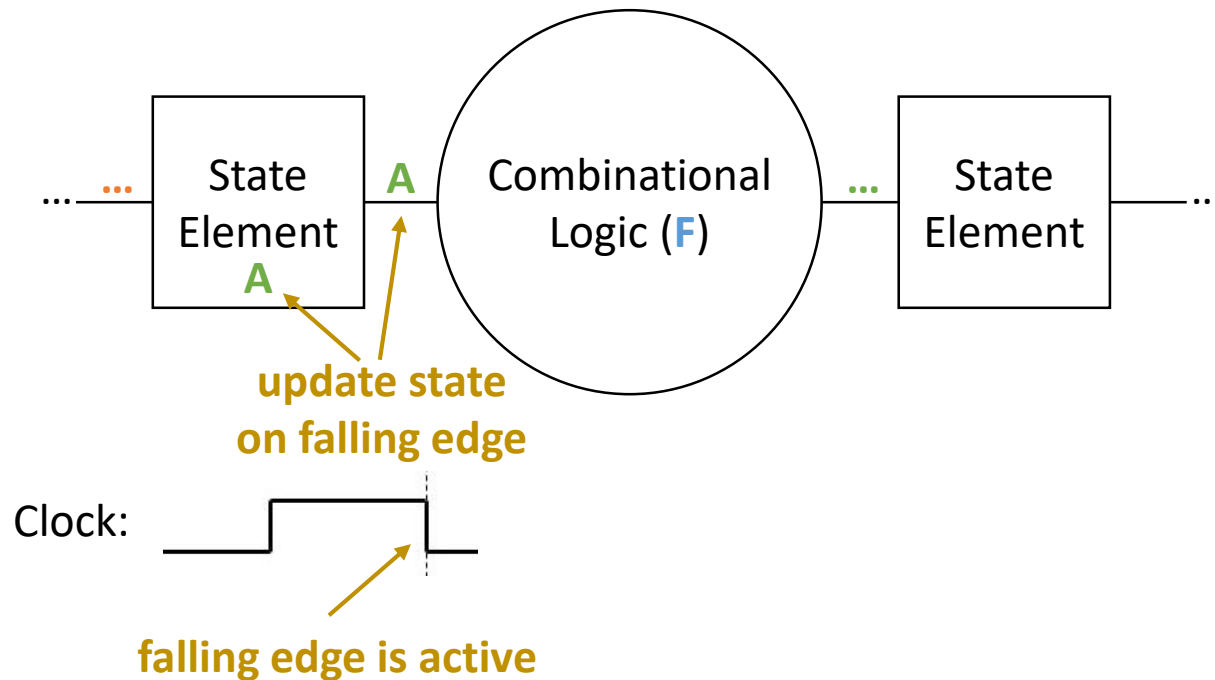


State Elements Example



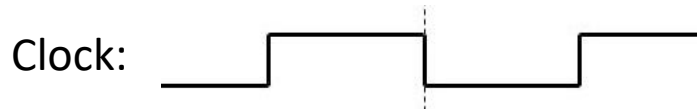
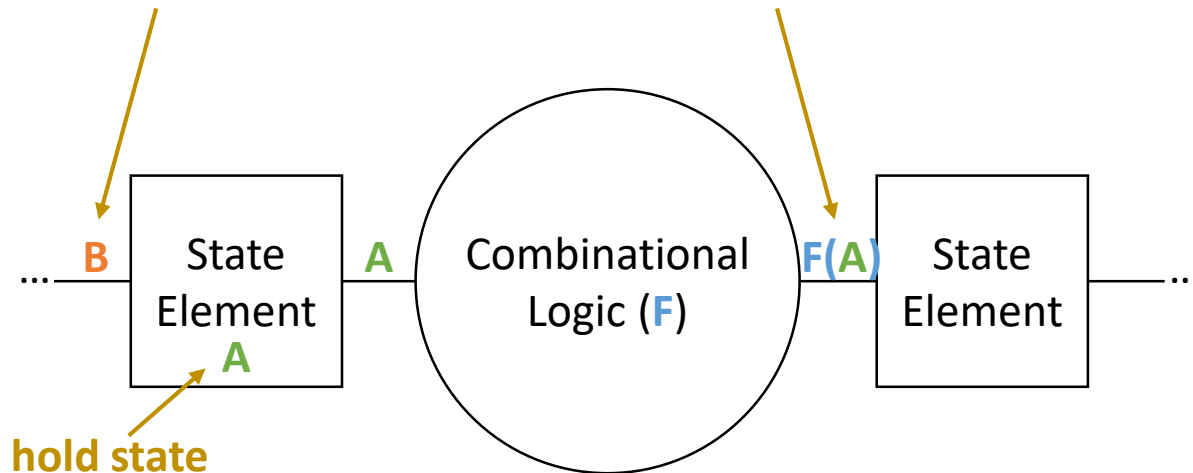
Clock: 

State Elements Example

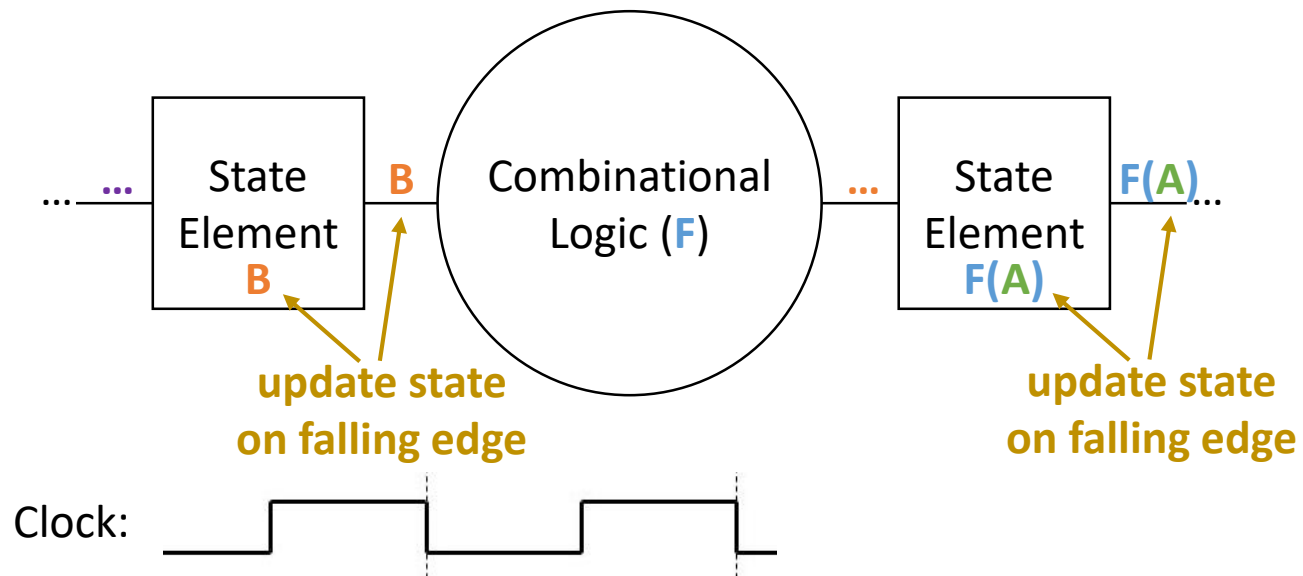


State Elements Example

during clock period, propagate through combinational logic to input of next state element

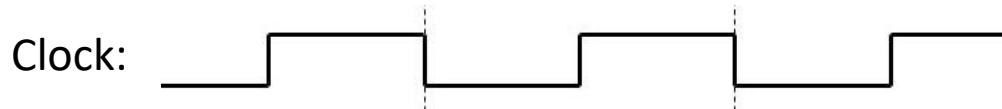
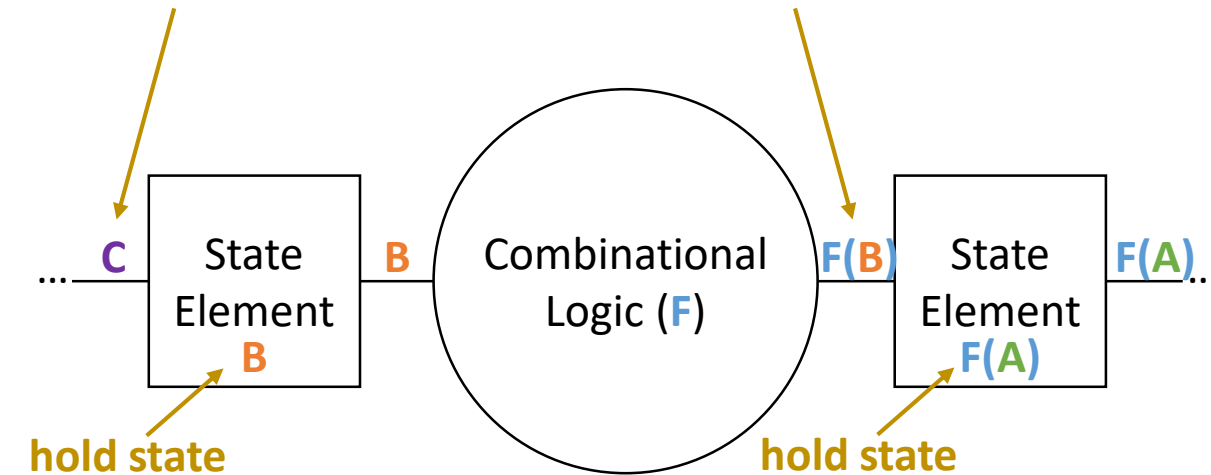


State Elements Example

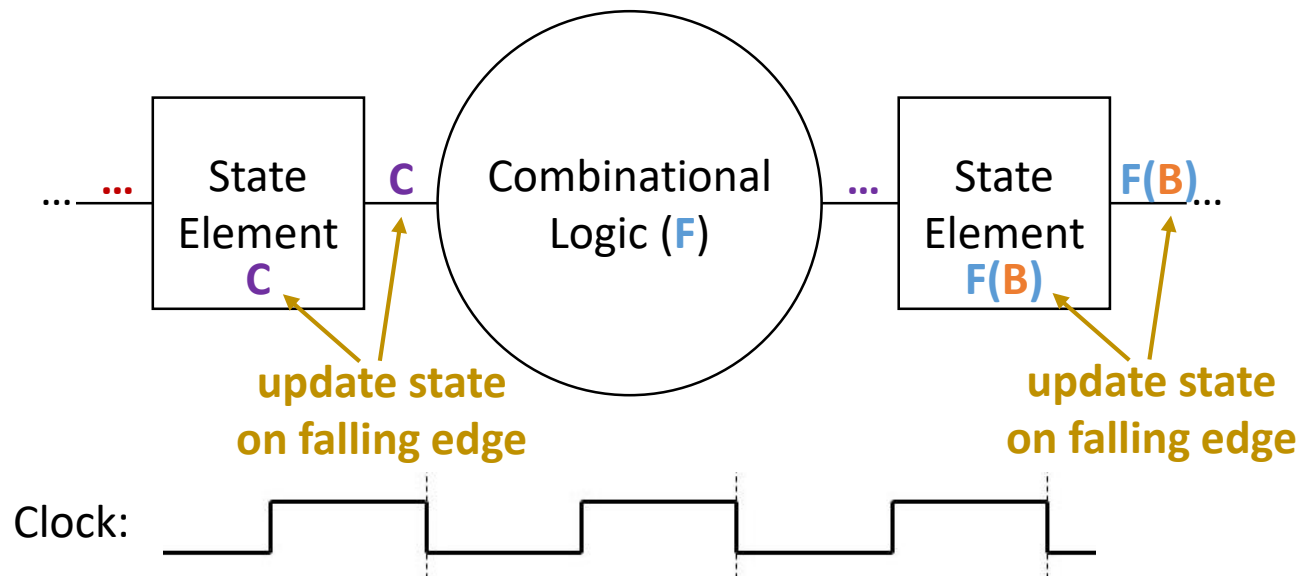


State Elements Example

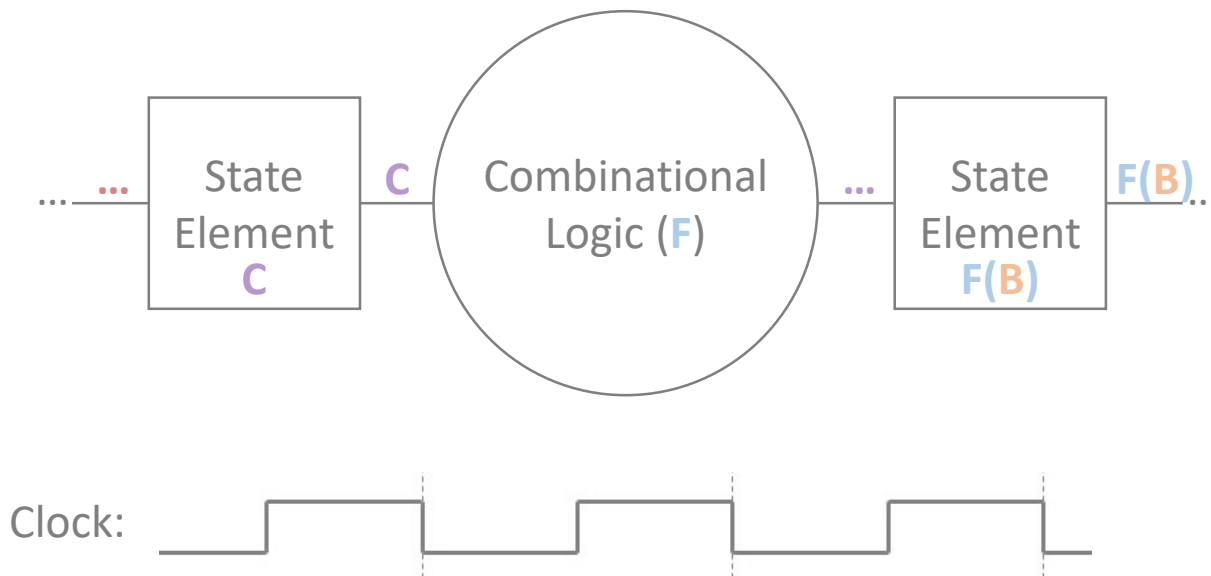
during clock period, propagate through combinational logic to input of next state element



State Elements Example



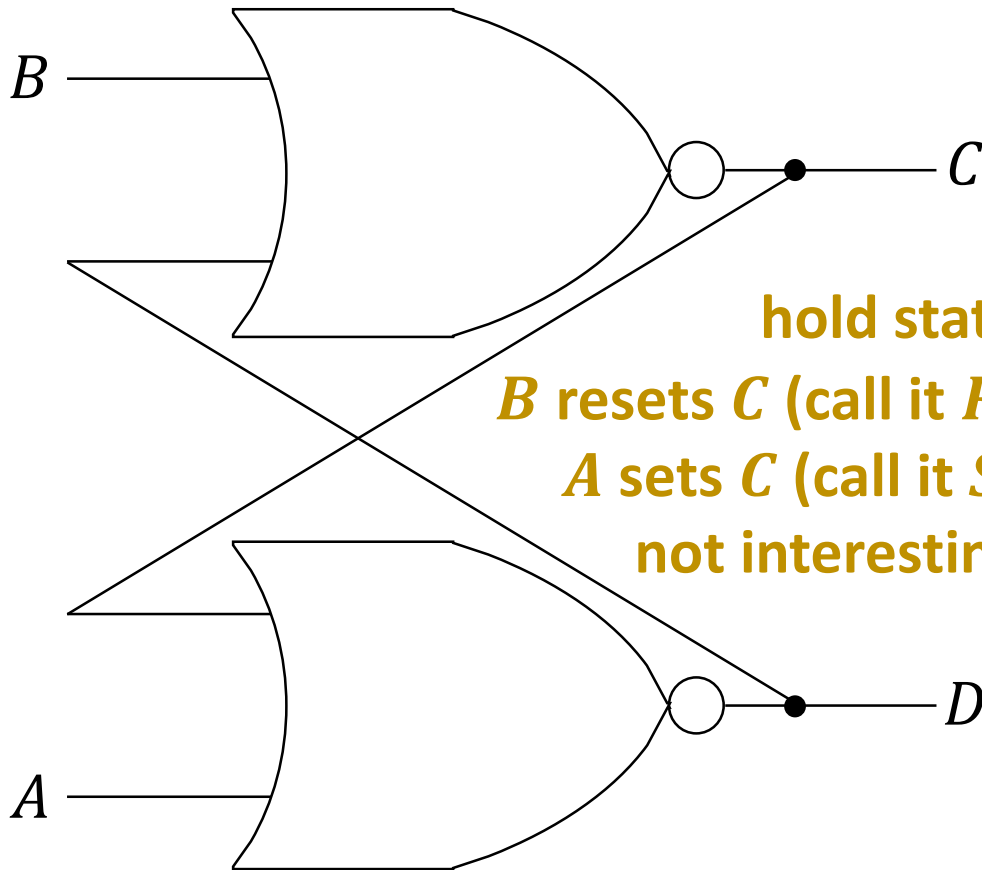
State Elements Example



How do we build a state element?

Goal: hold state throughout clock period and update state on falling edge

Latch



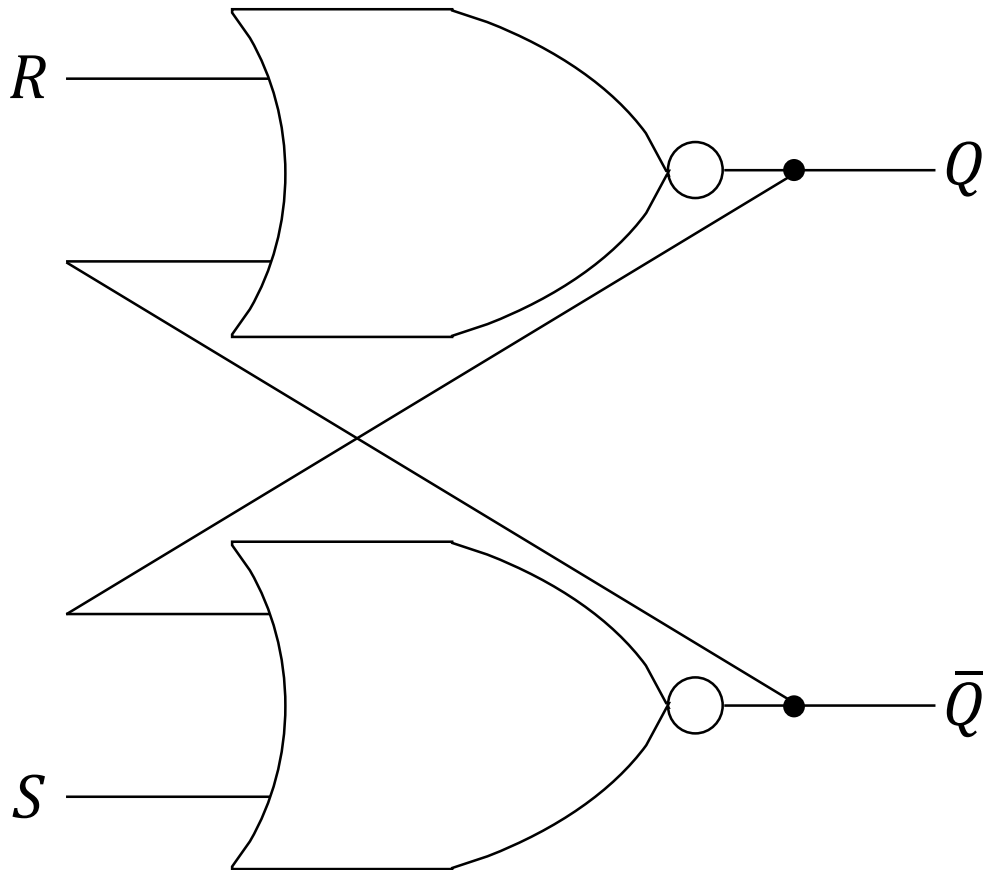
hold state →
 B resets C (call it R) →
 A sets C (call it S) →
not interesting →

D is $\bar{C}$ where it matters
(call them Q and $\bar{Q}$)

Inputs		Outputs	
A	B	C	D
0	0	old $\bar{D}$	old $\bar{C}$
0	1	0	1
1	0	1	0
1	1	0	0

Goal: hold state throughout clock period and update state on falling edge

SR Latch

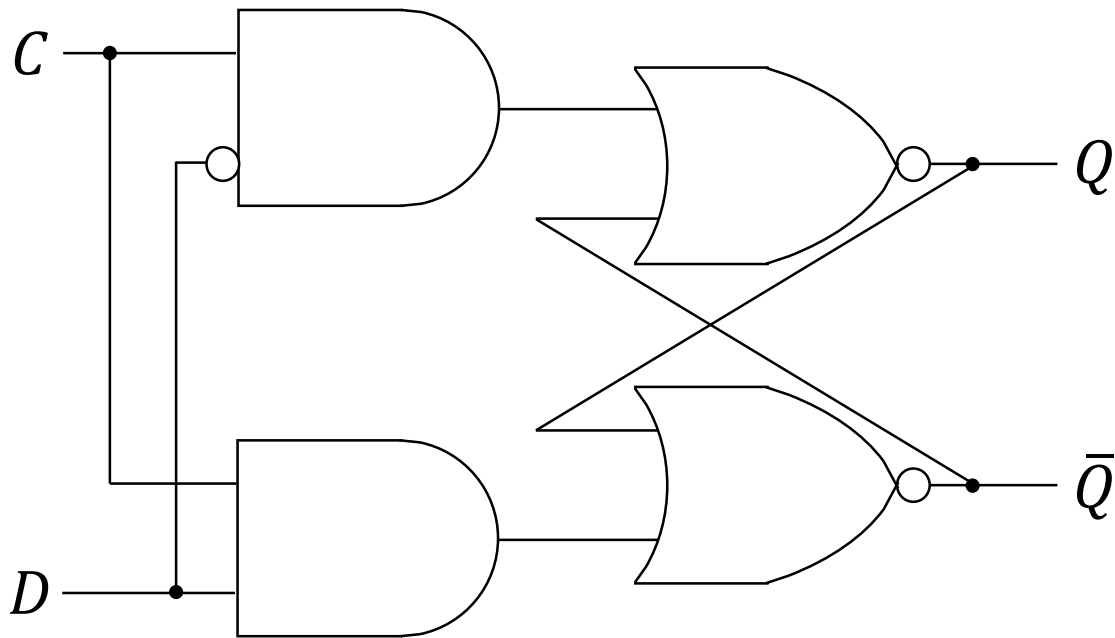


Inputs		Outputs	
S	R	Q	$\bar{Q}$
0	0	old Q	old $\bar{Q}$
0	1	0	1
1	0	1	0
1	1	0	0

hold state when SR is 00, update state to 0 when SR is 01,
update state to 1 when SR is 10

Goal: hold state throughout clock period and update state on falling edge

D Latch

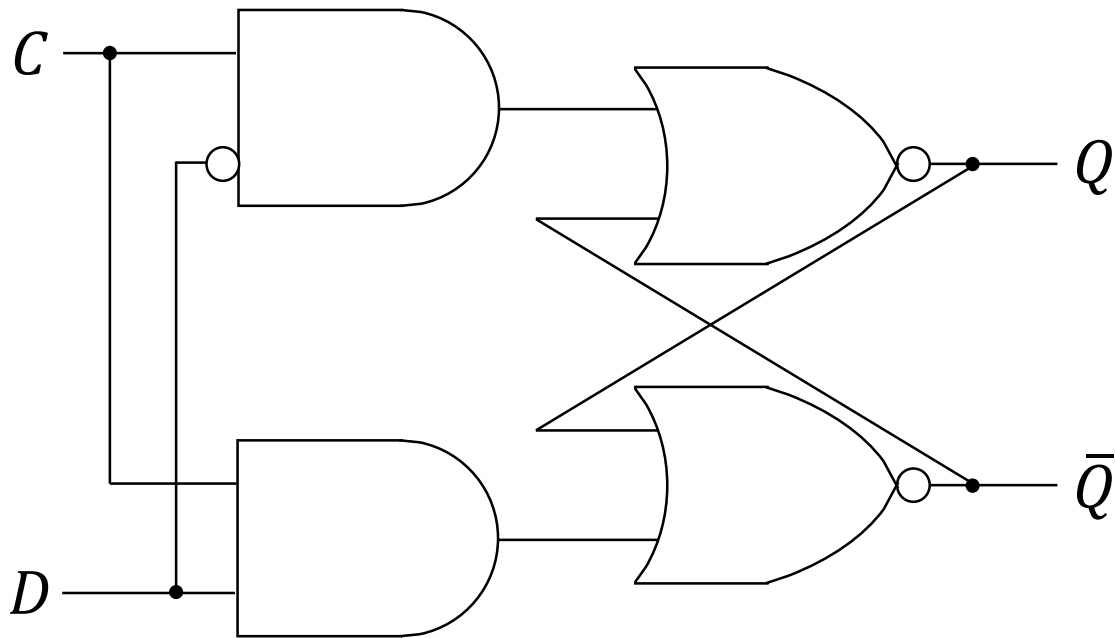


Inputs		Outputs	
C	D	Q	$\bar{Q}$
0	0	old Q	old $\bar{Q}$
0	1	old Q	old $\bar{Q}$
1	0	0	1
1	1	1	0

when C is 1, $Q = D$

Goal: hold state throughout clock period and update state on falling edge

D Latch

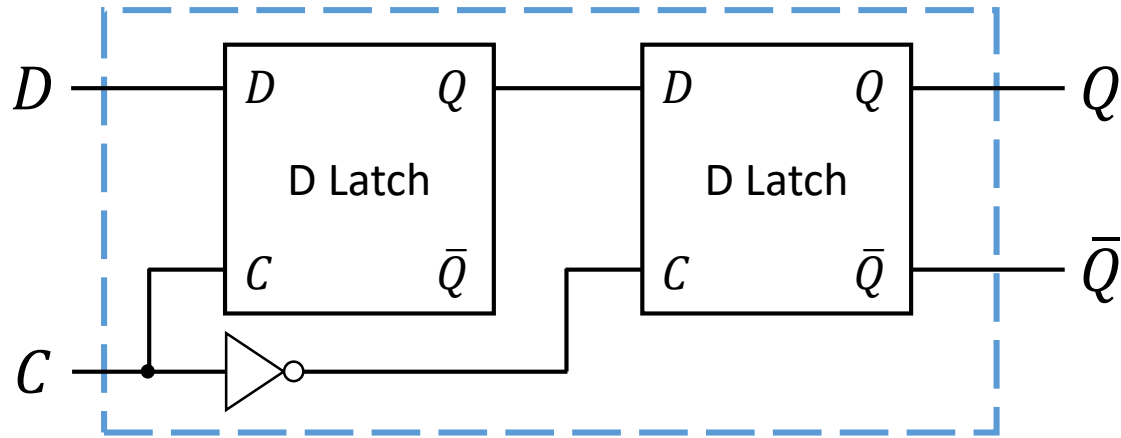


Inputs		Outputs	
C	D	Q	$\bar{Q}$
0	0	old Q	old $\bar{Q}$
0	1	old Q	old $\bar{Q}$
1	0	D	$\bar{D}$
1	1	D	$\bar{D}$

hold state when C is 0, update state to D when C is 1

Goal: hold state throughout clock period and update state on falling edge

D Flip-Flop

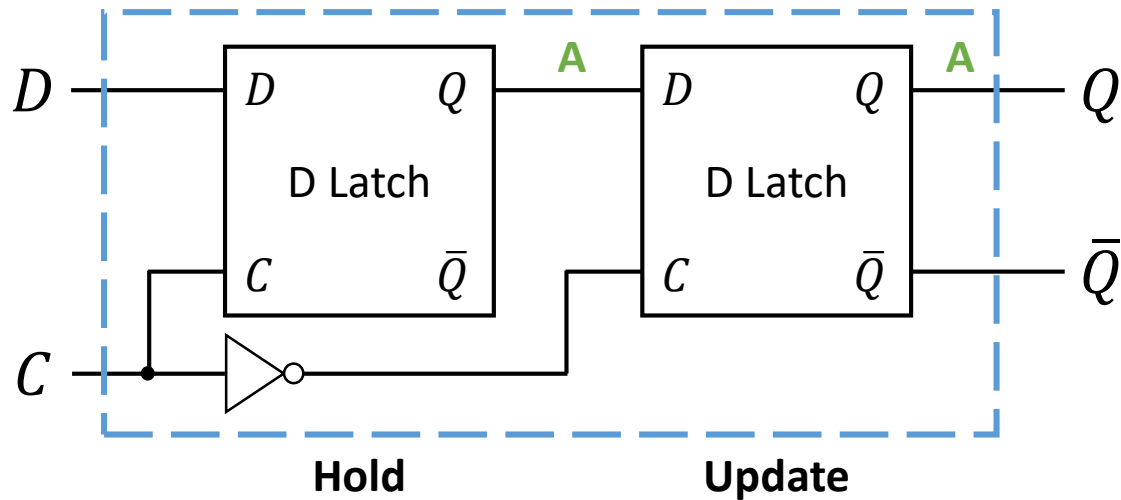


when C is 0, first latch holds state and second latch updates

when C is 1, first latch updates state and second latch holds prior state

Goal: hold state throughout clock period and update state on falling edge

D Flip-Flop Example



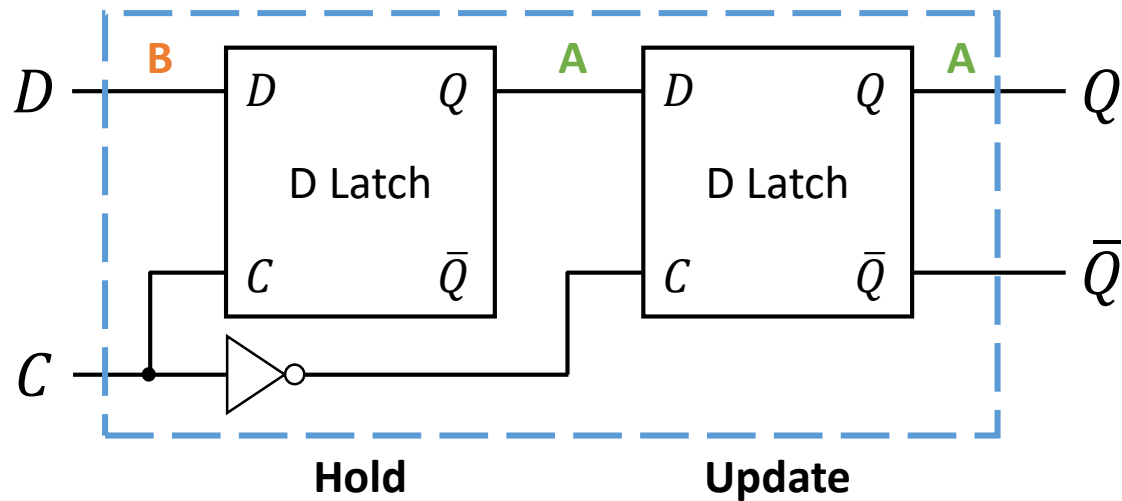
Clock (C): 

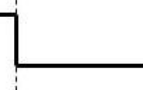
Input (D):

Output (Q): 

Goal: hold state throughout clock period and update state on falling edge

D Flip-Flop Example



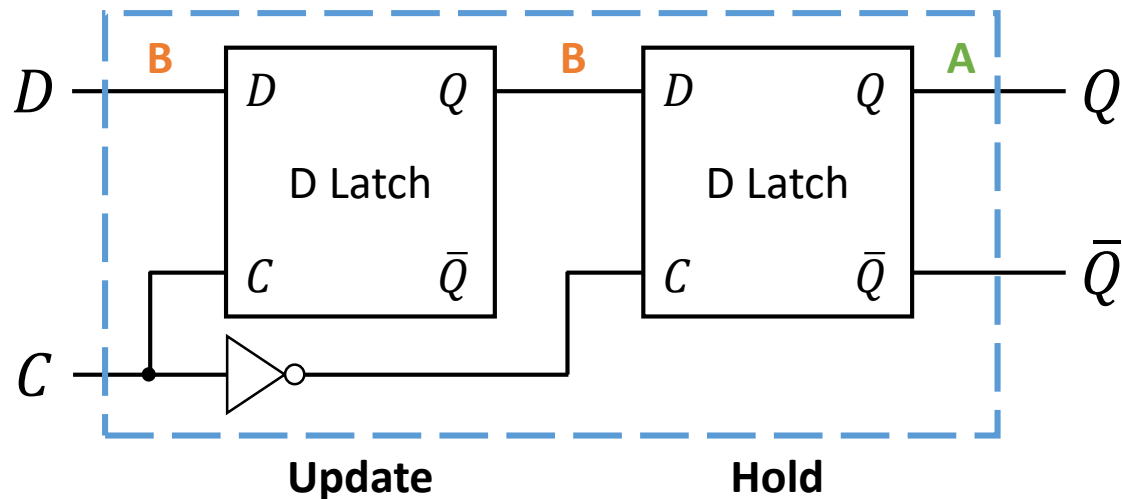
Clock (C): 

Input (D): 

Output (Q): 

Goal: hold state throughout clock period and update state on falling edge

D Flip-Flop Example



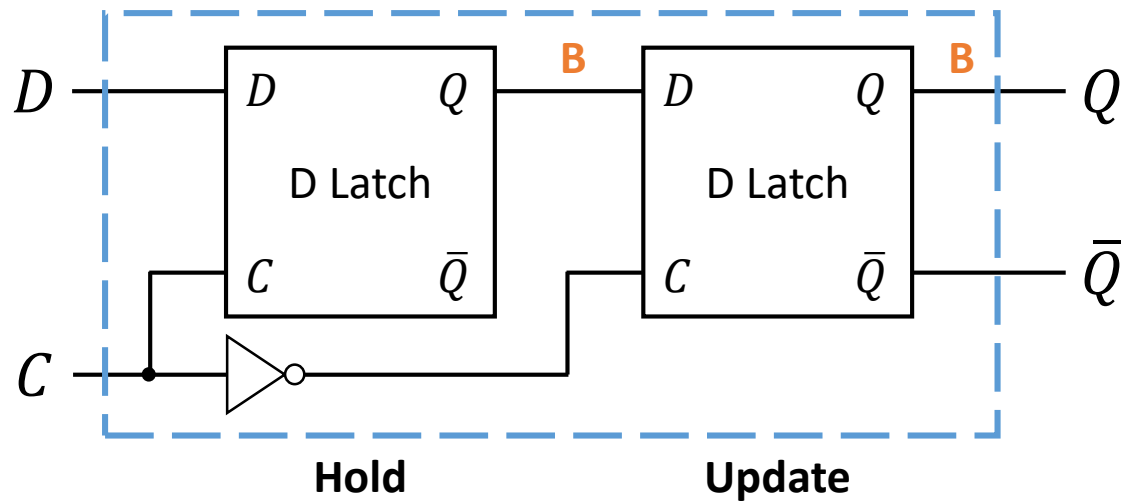
Clock (C):

Input (D):

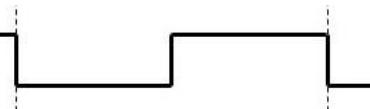
Output (Q):

Goal: hold state throughout clock period and update state on falling edge

D Flip-Flop Example



Clock (C):



Input (D):

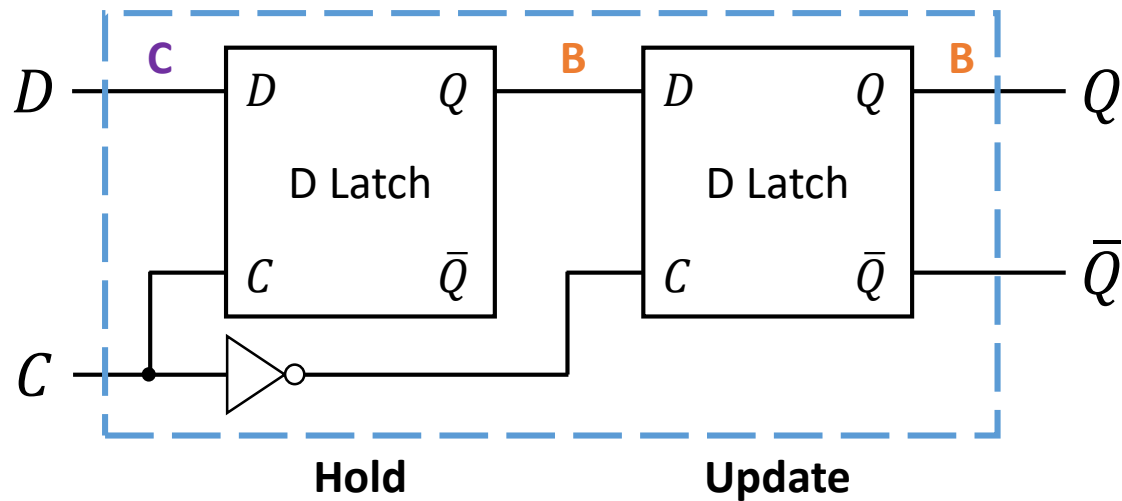


Output (Q):

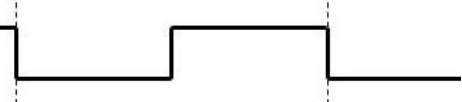


Goal: hold state throughout clock period and update state on falling edge

D Flip-Flop Example



Clock (C):



Input (D):

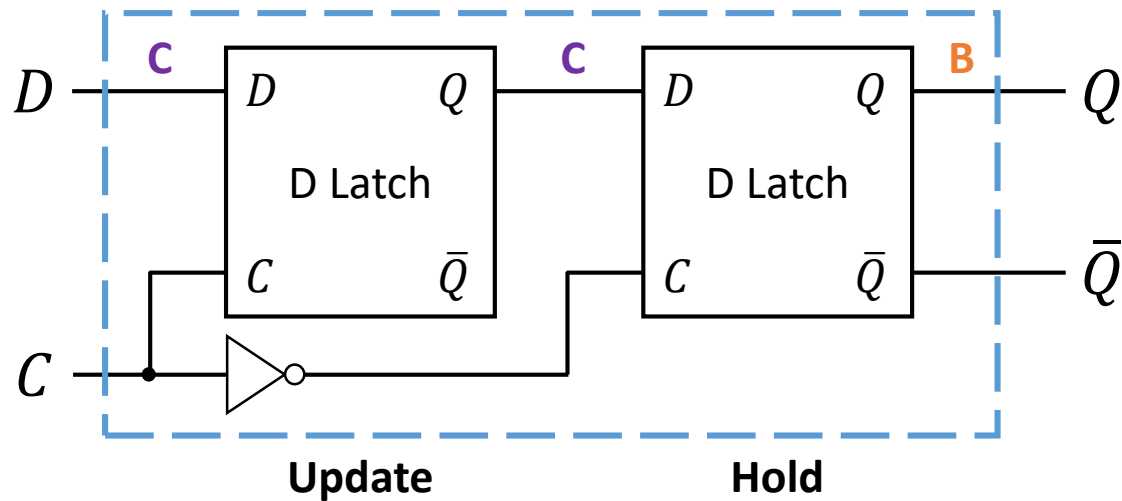


Output (Q):

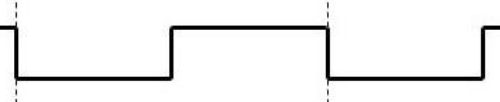


Goal: hold state throughout clock period and update state on falling edge

D Flip-Flop Example



Clock (C):



Input (D):

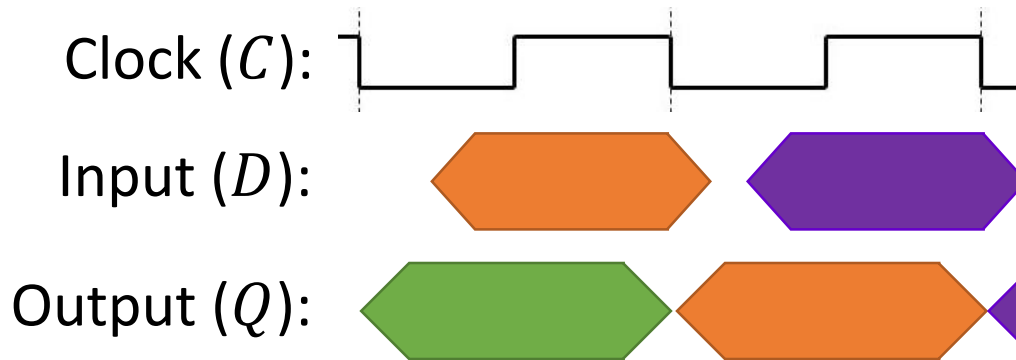
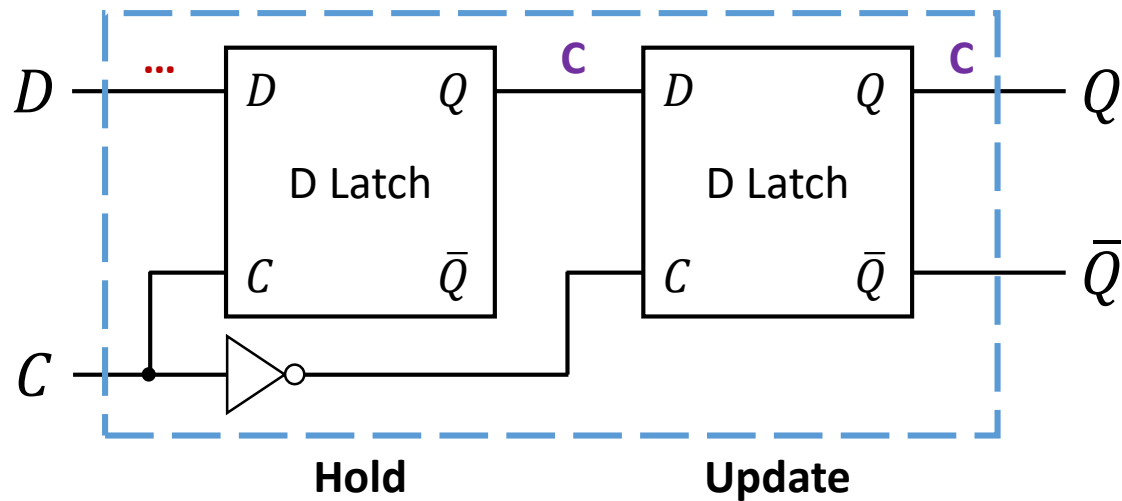


Output (Q):



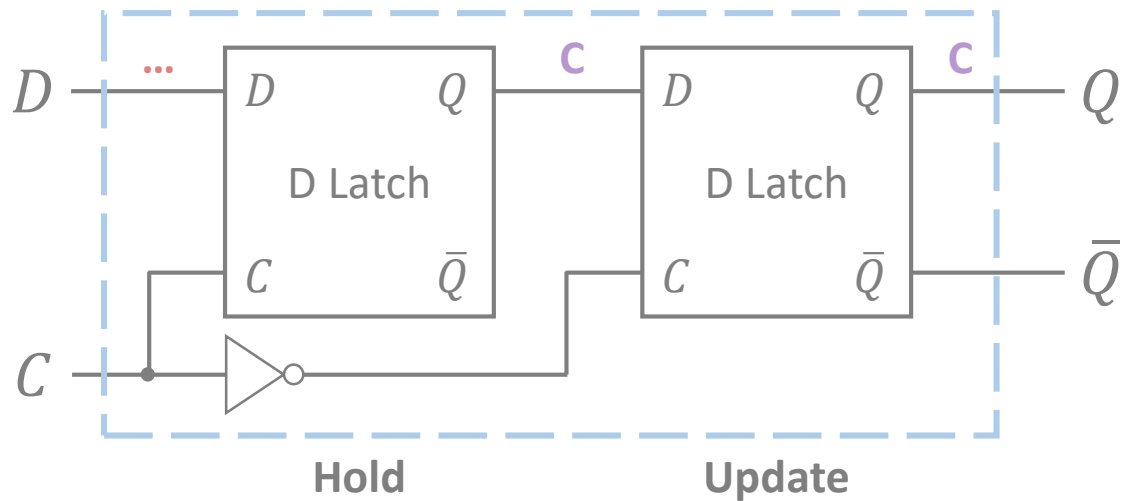
Goal: hold state throughout clock period and update state on falling edge

D Flip-Flop Example



Goal: hold state throughout clock period and update state on falling edge

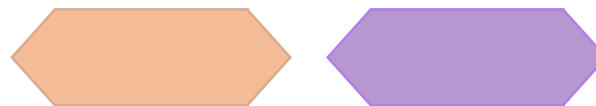
D Flip-Flop Example



Clock (C):



Input (D):



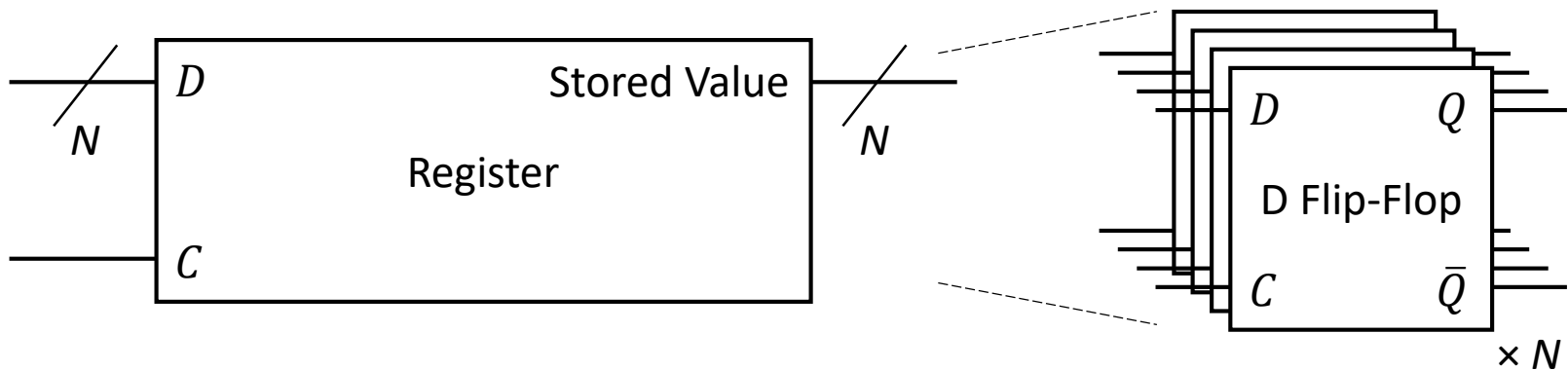
Output (Q):



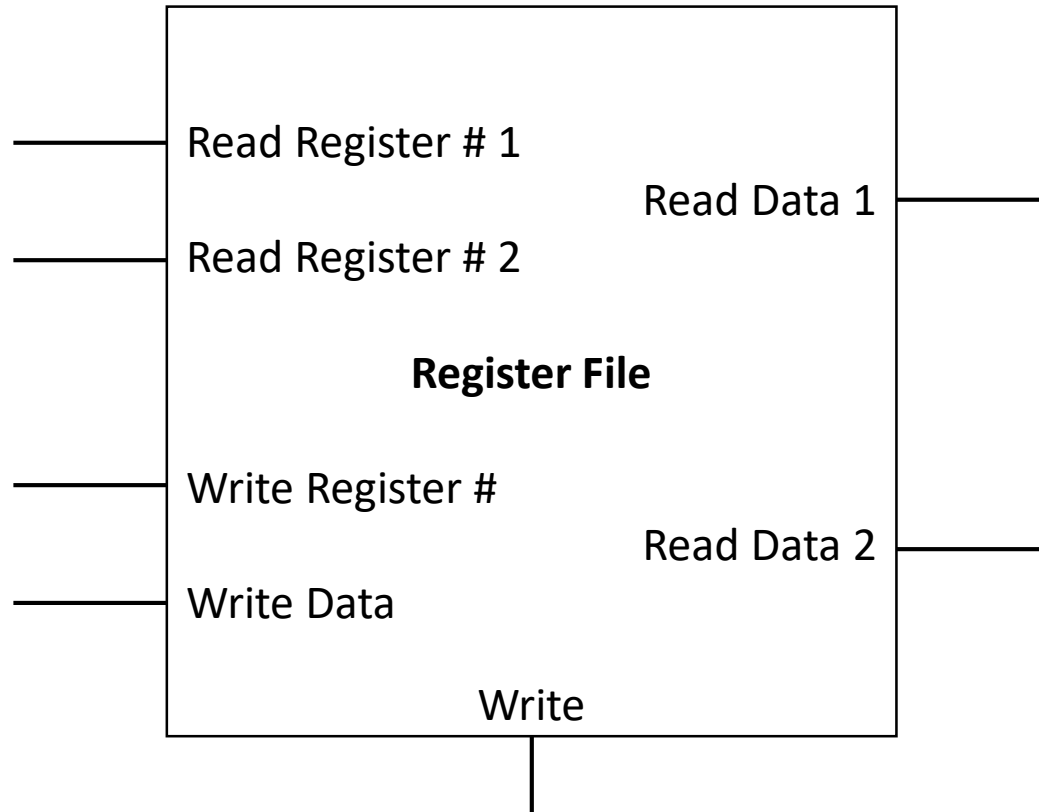
Goal: hold state throughout clock period and update state on falling edge ✓

Registers

- A D flip-flop acts as a 1-bit data storage cell
- A **register** is an array of N flip-flops, therefore acting as a N -bit data storage unit

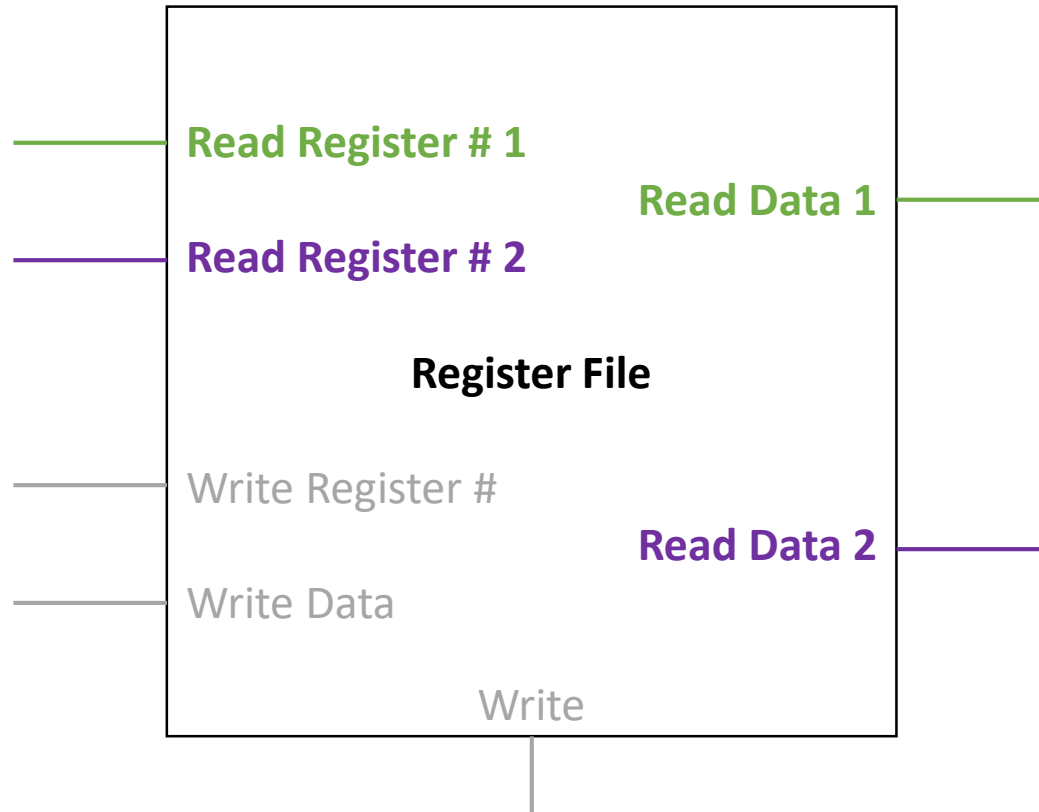


Register File



- A **register file** is a set of registers that each can be read and written by supplying the number of the register being accessed

Reading Registers

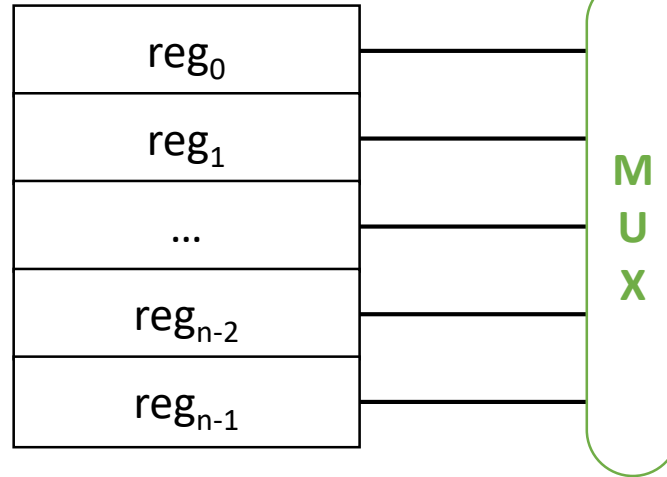


Reading Registers

reg_0
reg_1
...
reg_{n-2}
reg_{n-1}

Reading Registers

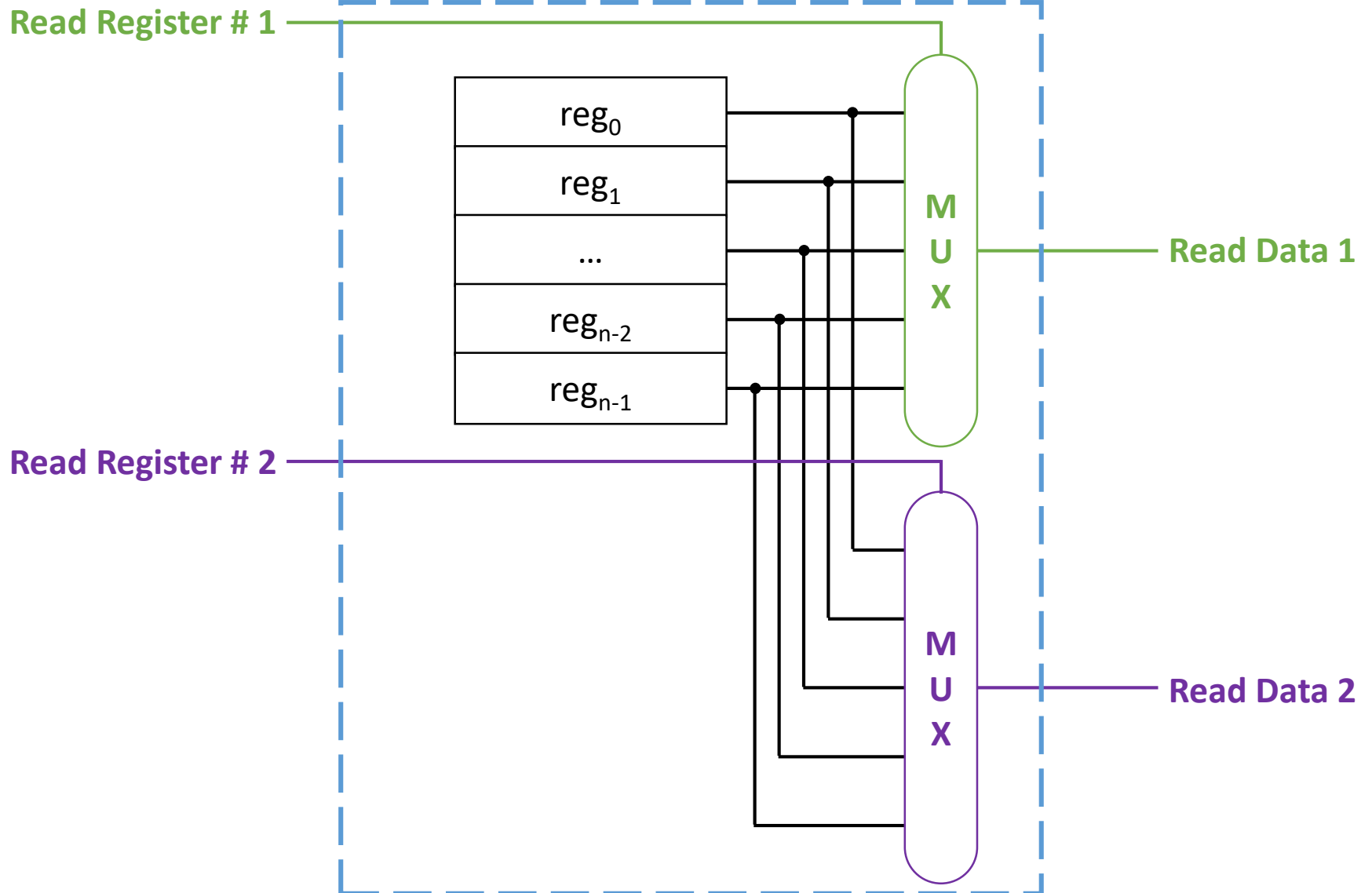
Read Register # 1



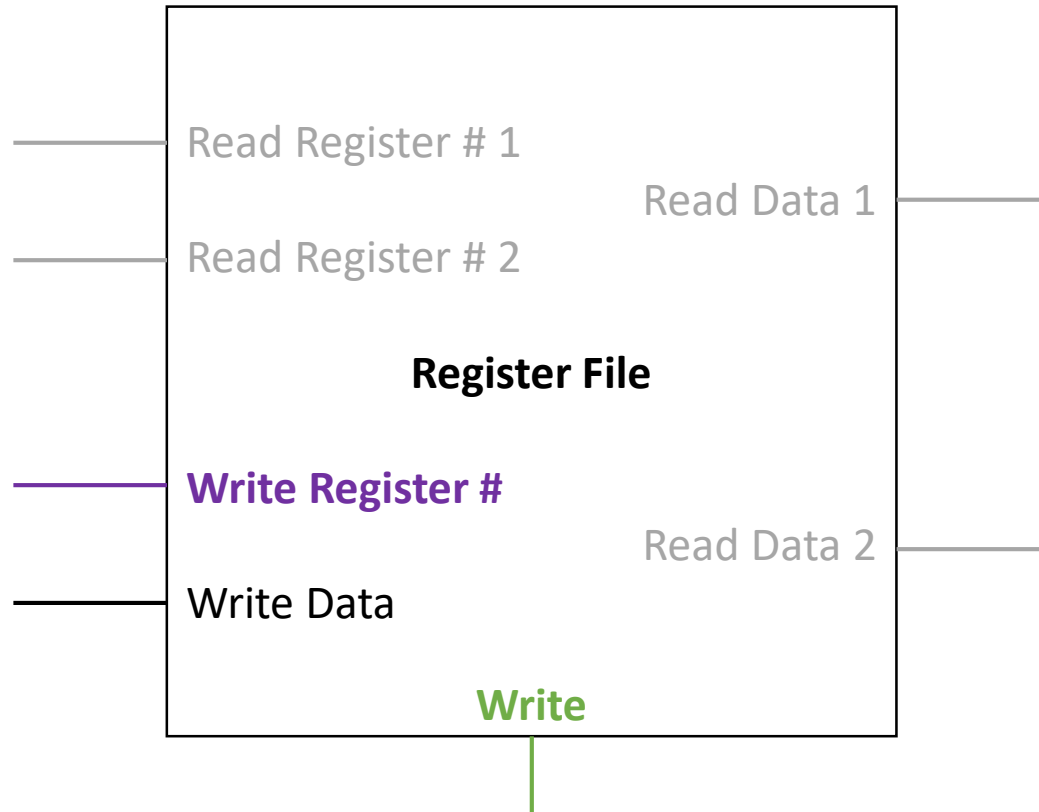
M
U
X

Read Data 1

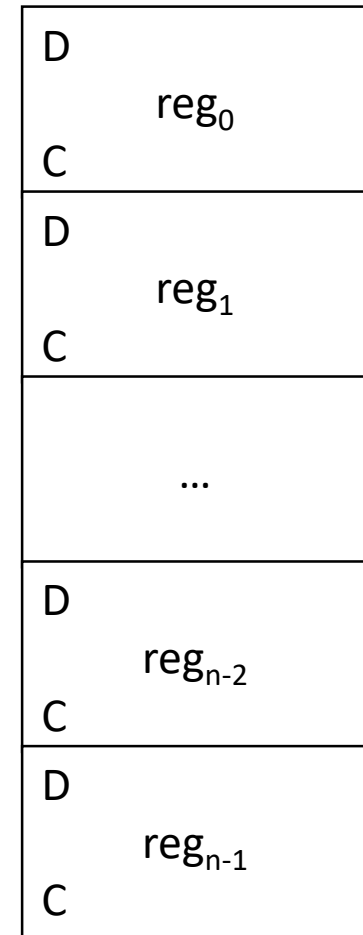
Reading Registers



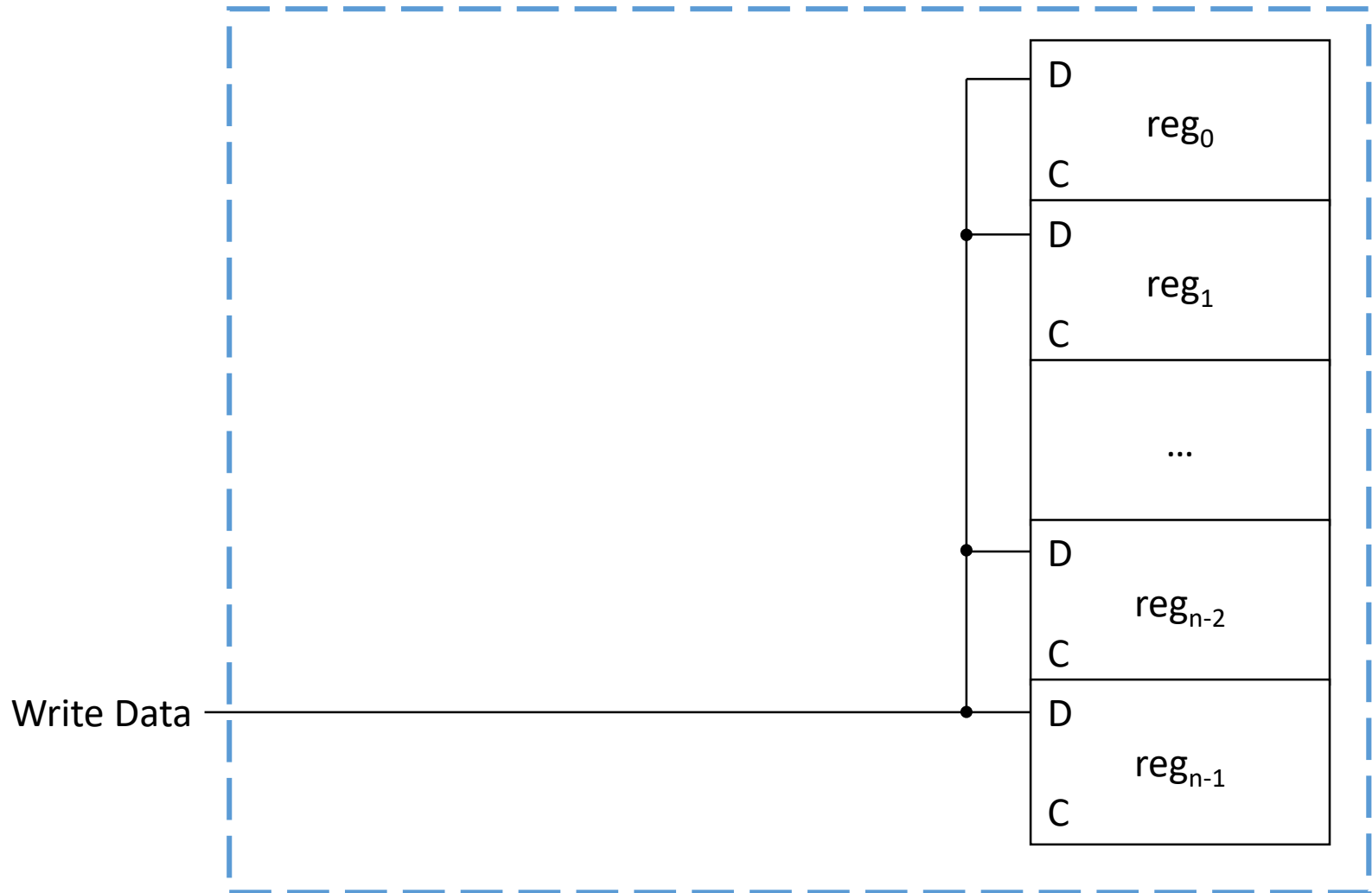
Writing Registers



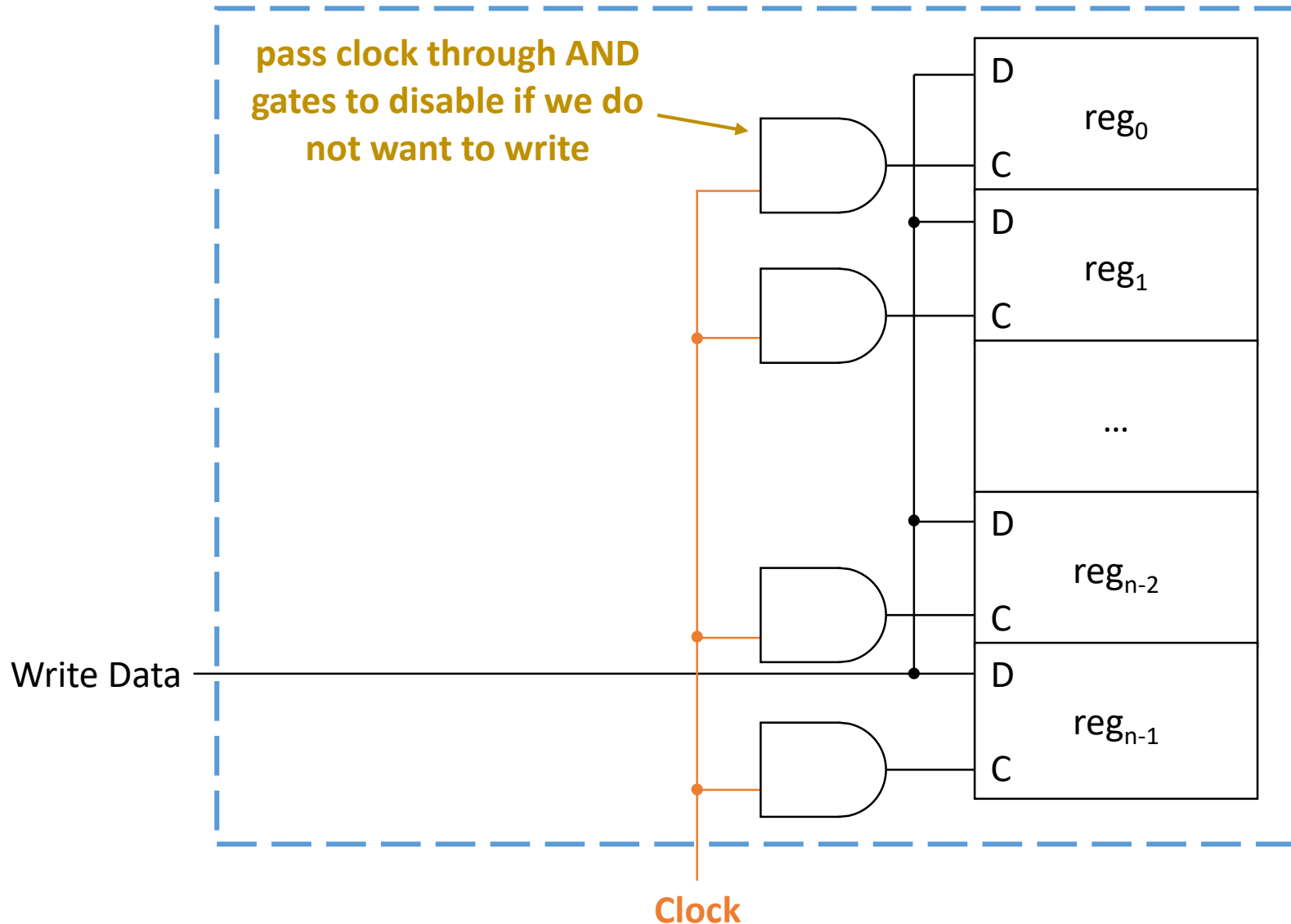
Writing Registers



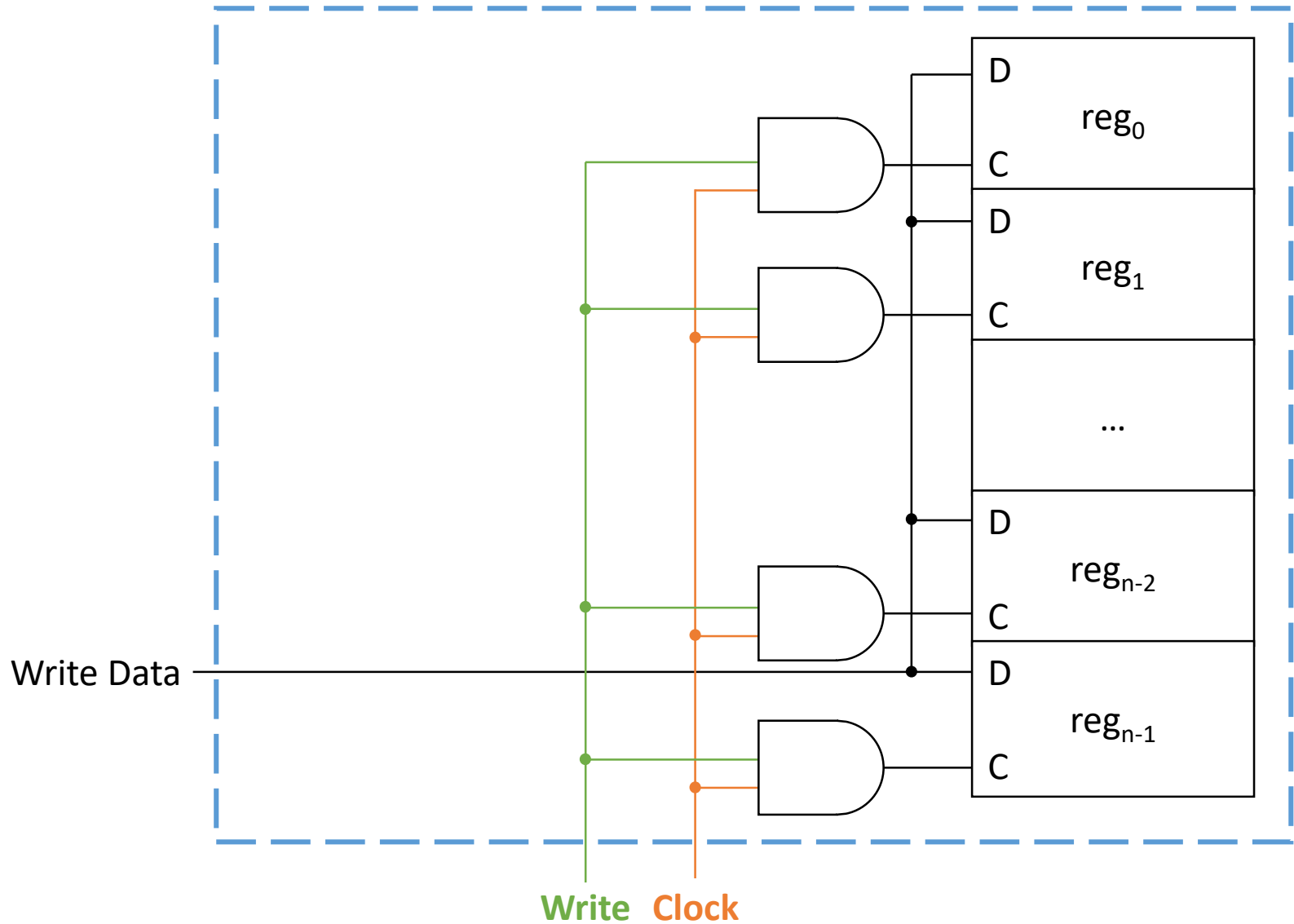
Writing Registers



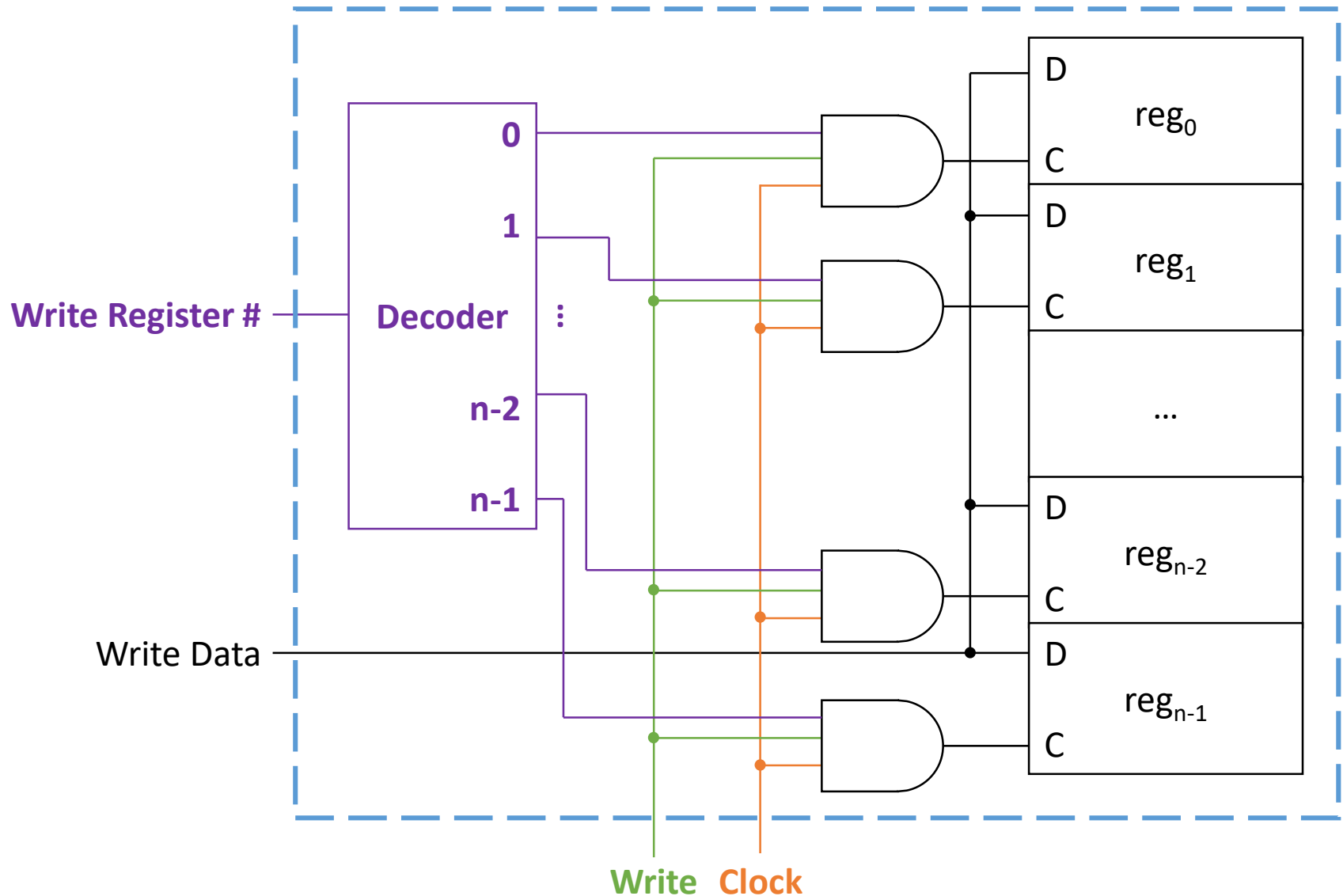
Writing Registers



Writing Registers



Writing Registers



Textbook Sections

- Some of the content in these slides corresponds to:
 - Textbook:
 - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
 - Sections:
 - B.7, B.8